

rSCI: Reconciled Software Carbon Intensity

PHILIPP WIESNER, Technische Universität Berlin, Germany

Metrics reported by cloud providers for Scope 3 GHG reporting are not the same metrics that help individual teams reduce emissions. Top-down provider accounting is auditable but coarse and delayed; bottom-up Software Carbon Intensity (SCI) signals are actionable but incomplete. We propose *rSCI* (reconciled SCI), a framework that bridges these two views: a per-workload carbon intensity metric that reconciles to the provider-reported total while preserving actionability. *rSCI* decomposes the gap between bottom-up estimates and top-down reports into per-scope residual bridges and allocates them back to individual workloads. We show that *rSCI* avoids the sunk carbon fallacy, rewards scheduling toward lower-overhead infrastructure, and degrades gracefully to *oSCI* when no provider data is available.

ACM Reference Format:

Philipp Wiesner. 2026. *rSCI: Reconciled Software Carbon Intensity*. 1, 1 (April 2026), 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Screen

$$SCI = C/R \quad (1)$$

2 Introduction

Metrics reported by cloud providers for Scope 3 GHG reporting are not the same metrics that help individual teams reduce their emissions [4].

Top-down methodology (reporting). The cloud provider computes its total monthly footprint from utility bills, diesel consumption, refrigerant losses, and amortized embodied carbon of hardware and buildings. It then allocates this total down to individual customers, services, and jobs. This data is auditable (backed by invoices), comparable (not yet due to underspecified parts in the GHG protocol, but hopefully eventually), but coarse (usually one number per customer $\times$ service $\times$ region $\times$ month) and delayed (available 2–3 weeks after month-end). It tells you the size of the problem but not what to do about it.

Bottom-up methodology (action). We measure (or model based on proxies) workload energy as close to the hardware as possible, combine it with location-based carbon intensity, and report an SCI (Software Carbon Intensity) [5] signal in near real time. This gives teams an optimization signal they can act on, but by itself it does not fully capture facility overhead, idle/shared capacity, provider-specific allocation effects, or all Scope 1/3 components.

These two signals cannot be identical. **But they must be reconcilable:** if a team improves its bottom-up signal, that improvement should be traceable through to the cloud provider's reported number at the reporting boundary the provider actually exposes. If the two diverge without explanation, users lose trust in both.

We focus on **location-based accounting**. Market-based accounting is decoupled from physical consumption and therefore weak as an energy/carbon efficiency signal.

2.1 Why Optimize Toward Imperfect Accounting Methodologies?

In practice, cloud customers' Scope 3 GHG accounting happens through the numbers providers report. Those numbers are imperfect and provider methodologies remain heterogeneous and partially opaque. Better standards are still needed, but even under imperfect reporting, users need

Author's Contact Information: Philipp Wiesner, Technische Universität Berlin, Germany, philipp.wiesner@tu-berlin.de.

2026. ACM XXXX-XXXX/2026/4-ART
<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Table 1. Comparison of SCI metric variants.

Metric	C	Includes	Key limitation
SCI	$E \cdot I + M$	Bottom-up operational + embodied	Sunk carbon fallacy
oSCI	$E \cdot I$	Bottom-up operational only	Misses embodied & overheads
tSCI	$E \cdot I + O_{\text{idle}} + M + M_{\text{idle}}$	SCI + idle overhead models	Requires DC knowledge
rSCI	$E \cdot I + w \cdot \Delta$	Operational + residuals	(this work)

a framework that maps actionable signals to the provider-reported number that governs reporting practice.

Goals of this framework:

- **For users:** provide an SCI-style metric per workload that sums up to the provider's delayed GHG report and makes uncertainties explicit.
- **For the ecosystem:** make GHG methodology quality visible by showing which energy/carbon efficiency optimization levers are rewarded, only approximately rewarded, or structurally blocked by provider accounting choices. This can inform/pressure better provider practices.

3 Motivation

The GSF's Software Carbon Intensity [5] was designed as an actionable optimization signal:

$$\text{SCI} = C/R, \quad (2)$$

which describes the carbon C per unit of work R (e.g., per request, per hour, per GB). In the original formulation, carbon is the sum of operational carbon O (energy E times carbon intensity I) and amortized embodied carbon M :

$$C = E \cdot I + M. \quad (3)$$

Bashir et al. [2] showed that including M can lead to perverse scheduling outcomes where carbon-aware schedulers route work to older, less efficient servers. They proposed:

- **oSCI** (operational only), which is the current state of the art for actionable carbon metrics and, according to the authors, the best metric for carbon-aware scheduling.
- **tSCI** (where overhead of idle servers is attributed to O and M), but they consider it impractical because it requires comprehensive datacenter-level information that cloud customers do not have access to.

We extend the taxonomy with **rSCI** (reconciled SCI): a variant that reconciles to the provider-reported total. Table 1 compares the metrics.

where Δ is the residual bridge and w the per-workload weighting factor.

3.1 Benefits of rSCI

- Shares tSCI's goal of allocating the full carbon footprint back to individual jobs, but is practical: learns $\Delta_{s,r}^{S2}$ from historical $(O, C^{\downarrow S2})$ pairs; Δ^{S1} and Δ^{S3} come directly from the provider report.
- More comprehensive: captures *everything* between the bottom-up total and the provider-reported number, separated by scope.
- Does **not** suffer from the sunk carbon fallacy: embodied carbon enters via $\Delta_{s,r}^{S3}$ and is distributed by energy share $(E_{i,s,r}/E_{s,r})$, not attributed per-server. Old hardware $\rightarrow$ higher $E \rightarrow$ more residual $\rightarrow$ always looks worse.

Table 2. Resource types and energy coefficients.

Resource type	Unit (q_p)	ε_p	Note
vCPU	vCPU-hours	Wh/vCPU-h	Reservation-based
GPU	GPU-hours	Wh/GPU-h	Reservation-based
Storage	GB-hours	Wh/GB-h	Reservation-based
Disk I/O	IO operations	Wh/IO-op	Event-based
Network transfer	GB transferred	Wh/GB	Event-based

- **Does** reward scheduling toward lower-overhead infrastructure: $\Delta_{s,r}^{S2}$ and $\Delta_{s,r}^{S3}$ differ across services and regions. oSCI is blind to these factors.

3.2 Boundary Mismatch Assumption

We assume the observable boundary covers all activity in the reporting slice (no boundary mismatch). A provider may bundle supporting services (storage I/O, managed networking) into a slice you only partially observe.

For example, you instrument all EC2 instances in eu-central-1, but the provider's Compute slice also includes EBS volume activity and NAT Gateway traffic.

- If we miss modeling an activity contributing to a top-down slice, the residual loses its interpretation. The additional unobserved activity will falsely be attributed to observed activities.
- If we miss modeling the contributions of a bottom-up activity to a top-down slice, the residual will falsely be attributed to observed activities.

4 rSCI Definition

Let i index **workload instances**—one evaluation of a workload over its period t_i . All per-instance quantities (E_i , C_i , R_i) are defined over one i . There are two common workload types:

- **Batch**: discrete jobs with a clear start and end where t_i is the job duration and R_i is a job-level quantity, e.g., 1 (the job), GB processed, or batches trained.
- **Interactive**: continuous services where t_i is a fixed window (e.g., 5 min) and R_i is a request-level quantity, e.g., number of requests, tokens, or queries.

4.1 Energy Model

We define the bottom-up energy estimate for workload i within service s and region r as:

$$E_{i,s,r} = \sum_p q_p \cdot \varepsilon_p \quad [\text{Wh}] \quad (4)$$

where p indexes resource types, q_p is quantity consumed during t_i , and ε_p is the energy coefficient (Wh/unit). The workload-level total is $E_i = \sum_{s,r} E_{i,s,r}$.

Table 2 lists the resource types and their units.

Not all resource types are observable. Unobservable components increase the residual.

This is a usage-based energy model: E depends on what you have provisioned or consumed ($q \cdot \varepsilon$), not on how hard you drive the resource. No provider currently rewards utilization at the customer boundary, so the utilization term does not help reconciliation. Utilization improvements are real (they reduce physical energy) but invisible to provider accounting (they fall into Δ^{S2}).

4.2 Residual Bridges Δ

Providers report Scope 1, 2, and 3 separately per **reporting slice** (e.g., GCP: project $\times$ service $\times$ region $\times$ month). We assume a workload does not span multiple projects and optimize within a single month (real-time scheduling signal).

Reporting slice granularity varies by provider:

- **GCP**—per project $\times$ service $\times$ region $\times$ month. Services are explicit (Compute Engine, Cloud Storage, BigQuery, etc.) [3].
- **Azure**—per subscription $\times$ resource type $\times$ region $\times$ month. Reports at individual resource granularity (the finest of the three providers) [6].
- **AWS**—per account $\times$ service $\times$ region $\times$ month. Only 3 named services: EC2, S3, CloudFront. All other services are aggregated into “Other” [1].

We index the top-down per-scope reported emissions $C^{\downarrow S1}$, $C^{\downarrow S2}$, and $C^{\downarrow S3}$ by service s and region r . The residual bridges are:

- $\Delta_{s,r}^{S1} = C_{s,r}^{\downarrow S1}$ (Scope 1) is fully residual: diesel, refrigerants.
- $\Delta_{s,r}^{S2} = C_{s,r}^{\downarrow S2} - O_{s,r}$ (Scope 2 gap) is the power usage we cannot estimate bottom-up: PUE, idle capacity, power model error.
- $\Delta_{s,r}^{S3} = C_{s,r}^{\downarrow S3}$ (Scope 3) is fully residual: embodied hardware/datacenter, FERA, upstream transport, etc.

where

$$O_{s,r} = \sum_i E_{i,s,r} \cdot I_r \quad (5)$$

is the bottom-up operational estimate. Carbon intensity I_r depends on the grid region.

4.3 Reconciliation

Reconciliation is performed per (s, r) . By default, all residuals are allocated by energy share:

$$w_{i,s,r} = E_{i,s,r} / E_{s,r} \quad (6)$$

We define:

$$\text{rSCI}_i = \frac{1}{R_i} \sum_{s,r} \left(O_{i,s,r} + w_{i,s,r} \left(\Delta_{s,r}^{S1} + \Delta_{s,r}^{S2} + \Delta_{s,r}^{S3} \right) \right) \quad (7)$$

Since $O_{i,s,r} = E_{i,s,r} \cdot I_r$, this collapses to:

$$\text{rSCI}_i = \frac{1}{R_i} \sum_{s,r} E_{i,s,r} \cdot \xi_{s,r} \quad (8)$$

where

$$\xi_{s,r} = I_r + \frac{\Delta_{s,r}^{S1} + \Delta_{s,r}^{S2} + \Delta_{s,r}^{S3}}{E_{s,r}} \quad (9)$$

is the **effective carbon intensity** of service s in region r . The only workload-specific term is $E_{i,s,r}$.

Since, by construction, $\sum_i w_{i,s,r} = 1$, it follows that $\sum_i \text{rSCI}_i \cdot R_i = \sum_{s,r} C_{s,r}^{\downarrow}$ over all observed (s, r) slices.

Bridge decomposition. Energy-proportional allocation is a reasonable default but may not be structurally optimal for all components (e.g., embodied carbon depends on hardware type, not energy). Each scope’s residual can be further decomposed into sub-components k with their own allocation keys $w_{i,s,r}^k$, as long as $\sum_i w_{i,s,r}^k = 1$:

- PUE overhead can meaningfully be allocated by energy share since it scales with energy.

- Idle capacity overhead may be better allocated by reservation size since it follows capacity held, not energy.
- Embodied carbon may be better allocated by instance-hours $\times$ hardware cost/weight.

5 Real-time Estimation

The retrospective rSCI requires Δ (from the provider report, delayed weeks) and $E_{s,r}$ (total slice energy, known only after the reporting period). To use rSCI as a real-time optimization signal, we estimate $\xi_{s,r}$ from the most recent reconciled month.

Why ξ is approximately stable across months. Providers allocate all overhead—embodied, idle, facility—proportionally to SKU usage. Both $C_{s,r}^\downarrow$ and $O_{s,r}$ therefore scale linearly with resource consumption, so $\xi_{s,r} = C_{s,r}^\downarrow / E_{s,r}$ is approximately load-independent.

Factoring out grid intensity. Grid carbon intensity I_r is volatile (varies hourly) but observable in real time and should not be estimated from last month. Decomposing ξ by scope:

$$\xi_{s,r} = \mu_{s,r} \cdot I_r + \rho_{s,r} \quad (10)$$

where:

- $\mu_{s,r} = C_{s,r}^{\downarrow S2} / (E_{s,r} \cdot I_r)$ is the **Scope 2 multiplier** (dimensionless; captures PUE and energy model error).
- $\rho_{s,r} = (C_{s,r}^{\downarrow S1} + C_{s,r}^{\downarrow S3}) / E_{s,r}$ is the **non-electricity overhead intensity** (gCO₂eq/Wh; Scope 1 + 3).

Both are slow-moving and load-independent. ρ can be split by scope for diagnostics if the provider reports Scope 1 and 3 separately.

The real-time estimate is then:

$$\widehat{\text{rSCI}}_i = \frac{1}{R_i} \sum_{s,r} E_{i,s,r} (\hat{\mu}_{s,r} \cdot I_r + \hat{\rho}_{s,r}) \quad (11)$$

where I_r is the current grid intensity and $\hat{\mu}, \hat{\rho}$ are from the last reconciled month.

Graceful degradation. Without any history, $\hat{\mu} = 1$, $\hat{\rho} = 0$, and rSCI reduces to oSCI.

6 Open Questions

- (1) **Energy coefficients.** Where to source reliable ε_p values per resource type / instance family / chip generation? Practical starting points include Cloud Carbon Footprint [7] (per-instance power estimates from SPECpower data), SPECpower benchmarks mapped to cloud SKUs, or GCP's per-project energy export for calibration. Note: absolute ε_p error is absorbed by μ (overestimate $\varepsilon \rightarrow$ higher $E \rightarrow$ lower μ ; reconciliation still holds). The real sensitivity is *relative* accuracy across resource types—if $\varepsilon_{\text{GPU}} / \varepsilon_{\text{CPU}}$ is wrong, workloads are compared on a skewed basis.
- (2) **Cold start and pooling.** Without historical data, $\hat{\mu}$ and $\hat{\rho}$ are unknown (rSCI degrades to oSCI). Since μ and ρ are driven by provider methodology (PUE, hardware mix), not individual customers, pooling per provider $\times$ service $\times$ region could provide priors for new customers.
- (3) **Non-stationarity.** μ and ρ can drift over time (seasonal PUE, hardware refreshes, methodology changes). How to detect regime shifts and update estimates?
- (4) **Provider methodology opacity.** rSCI reconciles to whatever the provider reports. A persistently large or volatile Δ^{S2} signals methodology weakness but cannot correct for it. How to distinguish “coarse provider methodology” from “missing observable activity”?

Should we provide a second, more methodologically complete estimate that makes visible the gap between what providers report and what an “ideal” methodology would report?

References

[1] Amazon Web Services. 2024. AWS Customer Carbon Footprint Tool Overview. <https://docs.aws.amazon.com/awsaccountbilling/latest/aboutv2/ccft-overview.html>.

[2] Noman Bashir, Fiodar Kazhamiaka, Christopher Engmann, Srinivasan Keshav, and Abel Radač. 2024. The Sunk Carbon Fallacy: Rethinking Carbon Footprint Metrics for Effective Carbon-Aware Scheduling. *arXiv preprint arXiv:2410.15087* (2024).

[3] Google Cloud. 2024. Google Cloud Carbon Footprint: Covered Services. <https://docs.google.com/carbon-footprint/docs/covered-services>.

[4] Green Software Foundation. 2024. Measuring for Reporting vs. Measuring for Action. <https://greensoftware.foundation/articles/measuring-for-reporting-vs-measuring-for-action>.

[5] Green Software Foundation. 2024. Software Carbon Intensity (SCI) Specification. <https://sci.greensoftware.foundation/>.

[6] Microsoft. 2024. Azure Carbon Optimization Overview. <https://learn.microsoft.com/en-us/azure/carbon-optimization/overview>.

[7] Thoughtworks. 2024. Cloud Carbon Footprint. <https://github.com/cloud-carbon-footprint/cloud-carbon-footprint>.